
第 4 章

Pandas 及其基本使用

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

4. Pandas 及其基本使用（新版讲义）

本页是第 4 章的章首页（索引）。正文位于 01-基础数据结构.md ~ 04-常见误区与技巧.md，与原版资料完全分离（原版见../原始资料/chap4/）。

本章概览

Pandas 是 Python 数据分析的事实标准：它以 Series（一维带标签数组）和 DataFrame（二维表格）两种核心结构，把 Excel/SQL 式的“表格数据操作”搬到编程环境里。它建立在 NumPy 之上，既保留向量化运算的高性能，又提供索引、筛选、分组、透视、缺失值处理、时间序列等丰富能力。本章先讲清两种基础数据结构的创建与索引，再介绍数据清洗（重复/缺失/异常值）、数据规约（Min-Max / Z-Score）、统计描述、分组聚合、透视表与时间序列，最后用综合案例把知识串起来。

学习目标

学完本章，你应该能够：

- 1. 熟练创建并操作 Series 与 DataFrame，理解 index、columns、dtype、shape 等属性。
- 2. 正确区分 .loc（按标签）与 .iloc（按位置），并能用布尔索引（&/|/~）做复杂筛选。
- 3. 完成数据清洗：去重（drop_duplicates）、缺失值处理（isna/fillna/dropna/ffill/bfill）、用 IQR 识别并处理异常值。
- 4. 掌握数据规约：Min-Max 归一化与 Z-Score 标准化，并能说出适用场景。
- 5. 会用 describe、groupby、agg、pivot_table、merge/concat 做统计、分组与透视分析。
- 6. 会用 date_range、resample、rolling 处理时间序列数据，并能用 matplotlib 把分析结果可视化。

案例索引

案例卡	所在节	技术点	难度
1. Series 索引对齐——为什么 s3 + s7 会出现 NaN	01 基础数据结构	Series 索引对齐、NaN、fillna/fill_value	★☆☆
2. groupby 分组统计——按班级算平均分并排名	02 数据分析	groupby、sort_values、rank	★★☆
3. 缺失值与异常值清洗——IQR 替换 + 时间序列重采样	02 数据分析	ffill、IQR、resample、rolling	★★☆
4. 学生成绩分析与可视化（综合）	03 综合案例	清洗 → 分组/透视 → 可视化	★★★

先修要求与运行环境

- 熟悉 Python 基础语法与 NumPy（第 1 章）；建议先学 [课程仓库参考（原：聪明办法学 Python）](#) 与第 1 章 numpy。
- 安装 Python 3.10+ 与 Pandas：

```
pip install pandas numpy matplotlib
# 需要读写 Excel 时再装
pip install openpyxl xlrd
```

- 推荐在 JupyterLab / VS Code 中打开.ipynb 练习；命令行列出的脚本直接保存为.py 运行即可。

本章目录

小节	文件	内容
01 基础数据结构	01-基础数据结构.md	Series 创建/访问/运算、DataFrame 创建/索引/切片/筛选、date_range + 案例卡 1（索引对齐）
02 数据分析	02-数据分析.md	重复/缺失/异常值、规约、统计描述、groupby、pivot_table、时间序列 + 案例卡 2/3
03 综合案例	03-综合案例.md	学生成绩分析：清洗 → 分组/透视 → 可视化（3 张配图）+ 案例卡 4（综合）
04 常见误区与技巧	04-常见误区与技巧.md	易错点表、loc/iloc 辨析、性能与调试、自测清单

数学预备与附录

本章的统计/时序背景已集中到附录《数学与算法补充》：

- **C 概率统计基础**：C.1 描述统计、C.5 时间序列基础（平稳性/差分/移动平均）；
- **F 通用速查**：F.4 数值稳定性；
- [打开附录 C](#) | [附录索引](#)

做分组聚合/透视前先看 C.1；第 7 章前可回看 C.5。

练习与上机入口

- [本章练习 \(exercises/\)](#)：作业题（15 题）、自测 quiz（25 题）、参考答案。
- [本章上机 \(lab/\)](#)：循序渐进的上机 notebook，含综合任务与检查清单。
- [本章参考与延伸阅读 \(references.md\)](#)：官方文档、精品教程、习题集、中文资料。
- [教学说明 \(teaching.md\)](#)：课时安排、重点难点、考核建议（教师用）。

建议课时

环节	学时	对应内容
讲课	3-4 学时	01-02 正文 + 03 案例讲解
上机	2-3 学时	lab/ 逐题完成；课后完成 exercises/ 作业

使用说明

- **学生**：先读 01-02 正文并运行代码 → 完成 lab 上机 → 提交 exercises 作业 → 自测 quiz 检验。遇到 loc/iloc、fillna 等细节时，回看 01/02 正文，或到 references.md 找官方章节。
- **教师**：按 teaching.md 的课时表讲；lab 可作为上机课内容；exercises 中的 quiz 已整理为 Pandas 专项，可直接回收；03 综合案例配图用于课堂演示。

01 基础数据结构：Series 与 DataFrame

本节对应原版 4.1 的内容，并增补学习目标、常见误区、思考题与延伸阅读。

本节目标

- 理解 Series（一维带标签）与 DataFrame（二维表格）两种核心结构；
- 会用列表、NumPy 数组、字典创建 Series 与 DataFrame，并指定 index/columns；
- 掌握 .loc（按标签）与 .iloc（按位置）的区别；
- 会用布尔索引 + & / | / ~ 做多条件筛选；
- 会用 sort_values、concat、merge 做排序与合并；
- 会用 date_range 生成日期索引，并按日期筛选。

先修

- Python 列表、字典、推导式；第 1 章 NumPy 数组概念 (ndarray、shape、dtype)。

官方文档 / 参考入口

- Pandas 官方入门：[链接](#)
- Pandas Series 参考：[链接](#)
- Pandas DataFrame 参考：[链接](#)

1.1 为什么用 Pandas

NumPy 的 ndarray 擅长向量化数值计算，但没有“列名/行名/标签”的概念，也缺少缺失值、分组、透视等表格能力。Pandas 在 NumPy 之上提供：

特性	NumPy ndarray	Pandas Series/DataFrame
维度	任意 N 维	1 维 (Series) / 2 维 (DataFrame)
标签	只有整数位置索引	可自定义 index (行) 与 columns (列)
缺失值	常需 np.nan + 手写	isna/fillna/dropna 成套
分组/透视	需手写	groupby/pivot_table
与 Excel/SQL 接近程度	低	高

1.2 Series

1.2.1 创建 Series

```
import pandas as pd
import numpy as np
np.set_printoptions(precision=4, suppress=True)
pd.set_option('display.width', 100)
```

```
pd.set_option('display.max_columns', 20)

s1 = pd.Series([1, 2, 3, 4, 5])           # 列表
s2 = pd.Series(np.array([1, 2, 3, 4, 5])) # NumPy 数组
s3 = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e'])
s4 = pd.Series({'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}) # 字典
print(s3)

a    1
b    2
c    3
d    4
e    5
dtype: int64
```

不指定索引时, pandas 自动生成从 0 开始的整数索引。

1.2.2 访问与修改

```
print(s3['a'])      # 按标签
print(s3.iloc[0])   # 按位置 (推荐, 避免整数歧义)

s3['a'] = 100        # 按标签修改
s3.iloc[0] = 100     # 按位置修改
print(s3)

1
1
a    100
b     2
c     3
d     4
e     5
dtype: int64
```

新版 pandas 中, 用 `s[0]` 取 “位置” 会触发弃用警告; 位置访问请统一 `s.iloc[0]`, 标签访问用 `s['a']`。

1.2.3 索引切片

```
print(s3['a':'c'])   # 基于标签的切片: 闭区间
s5 = pd.Series([1, 2, 3, 4, 5], index=[0, 1, 2, 3, 4])
print(s5[0:3])       # 整数索引时按位置切片

a    100
b     2
c     3
dtype: int64
0     1
1     2
2     3
dtype: int64
```

注意: 标签切片**包含终点** (比如 `'a':'c'` 含 c), 而 Python 列表切片不含终点。

1.2.4 运算与索引对齐

```
s6 = s3 + 10
print('s3 + 10:')
print(s6)

s7 = pd.Series([1, 2, 3], index=['a', 'c', 'e'])
print('s3 + s7:')
print(s3 + s7)      # 按共同索引对齐; 缺失处为 NaN

s3 + 10:
a      110
b       12
c       13
d       14
e       15
dtype: int64
s3 + s7:
a      101.0
b         NaN
c         5.0
d         NaN
e         8.0
dtype: float64
```

两个 Series 相加时, 索引会**对齐**; 没有共同索引的位置得到 NaN, 这是 Pandas 的重要语义。

1.2.5 常用属性与方法

```
print(s3.index)      # Index(['a', 'b', 'c', 'd', 'e'], dtype='object')
print(s3.values)     # array([100,  2,  3,  4,  5])
print(s3.describe()) # 统计摘要
print(s3.head(2))    # 头
print(s3.tail(2))    # 尾
print(s3.isnull())   # 是否缺失
print(s3.notnull())  # 是否非缺失

Index(['a', 'b', 'c', 'd', 'e'], dtype='object')
[100  2  3  4  5]
count      5.000000
mean       22.800000
std        43.170592
min         2.000000
25%         3.000000
50%         4.000000
75%         5.000000
max        100.000000
dtype: float64
a      100
b         2
dtype: int64
d         4
e         5
dtype: int64
a      False
```

```
b    False
c    False
d    False
e    False
dtype: bool
a    True
b    True
c    True
d    True
e    True
dtype: bool
```

小提醒：统计方法（describe/mean/std 等）会**自动忽略 NaN**；value_counts()（注意是复数 counts）统计每个取值出现次数。

案例卡 1：Series 索引对齐——为什么 s3 + s7 会出现 NaN

目标

理解 Pandas 的**索引对齐**：两个 Series 相加时，不是按“位置”相加，而是按**相同的标签**对齐；标签只在一边出现过时，结果那个位置就是 NaN（缺失）。这个案例让你亲眼看到 NaN 怎么产生，并学会用 fill_value=0 或 fillna(0) 把它消除。

代码

```
import numpy as np
import pandas as pd
pd.set_option('display.width', 100)
pd.set_option('display.max_columns', 20)
np.set_printoptions(precision=4, suppress=True)

s3 = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e'])
s7 = pd.Series([1, 2, 3], index=['a', 'c', 'e'])

print('s3:')
print(s3)
print('s7:')
print(s7)
print('s3 + s7 （索引对齐, 缺失为 NaN）:')
print(s3 + s7)
print('s3.add(s7, fill_value=0) （先补 0 再相加）:')
print(s3.add(s7, fill_value=0))
```

运行结果（已运行核验）

```
s3:
a    1
b    2
c    3
d    4
e    5
dtype: int64
```

```
s7:
a    1
c    2
e    3
dtype: int64
s3 + s7 （索引对齐，缺失为 NaN）：
a    2.0
b    NaN
c    5.0
d    NaN
e    8.0
dtype: float64
s3.add(s7, fill_value=0) （先补 0 再相加）：
a    2.0
b    2.0
c    5.0
d    4.0
e    8.0
dtype: float64
```

讲解

一句话解释：把两个带标签的 Series 相加时，Pandas 先“对上号（相同标签）再加法”；对不上号的位置没有值，就记作 NaN。

- s3 的索引是 a,b,c,d,e，s7 的索引是 a,c,e；
- s3 + s7 时，a、c、e 三处两边都有值，正常相加；b、d 只有 s3 有，另一边“没有这一格”，所以结果是 NaN；
- 因为结果里有 NaN，整个结果的 dtype 自动变成 float64（整数类型无法表示 NaN）；
- s3.add(s7, fill_value=0) 表示“缺失的一边按 0 参与运算”，所以 b 得到 2+0=2，d 得到 4+0=4；
- 另一种等价写法是 (s3 + s7).fillna(0)：先按索引对齐算出 NaN，再用 fillna(0) 把它们填成 0。

主要用法 / API

需求	写法	说明
两个 Series 相加（按索引对齐）	s3 + s7	缺失标签给 NaN
相加时缺失处补 0	s3.add(s7, fill_value=0)	add/sub/mul/div 都支持 fill_value
先算出 NaN 再补 0	(s3 + s7).fillna(0)	fillna 统一填充
判断是否 NaN	s.isna() / s.notna()	返回布尔 Series
只保留共同索引	(s3 + s7).dropna()	丢弃 NaN 位置

常见错误

- 以为 Series 加法是“按位置逐个加”，忽略了它是**按标签对齐**；
- 看到 NaN 以为代码出错——其实这是 Pandas 的**设计语义**，不是 bug；
- 混淆 s3.fillna(0) 与 (s3 + s7).fillna(0)：前者只填 s3 自身的缺失值，不是“把两个 Series 缺失补 0 再相加”；
- 相加出现 NaN 后直接 astype(int)，会因 NaN 报错；应先 fillna(0) 再转类型。

拓展 / 跟练

1. 试试 `s3.sub(s7, fill_value=0)` 和 `s3.mul(s7, fill_value=0)`, 观察输出差异。
2. 把 `s7` 的索引改成 `['a', 'b', 'c', 'd', 'e']`, 再看 `s3 + s7` 是否还有 `NaN`。
3. 用 `pd.concat([s3, s7], axis=1)` 把两个 Series 并成一个 DataFrame, 观察对齐与 `NaN`。
4. 用 `s3.reindex(['a', 'c', 'e', 'f'])` 手动规定索引, 再和 `s7` 相加, 体会 `reindex` 的作用。

1.3 DataFrame

1.3.1 创建 DataFrame

1) 二维列表

```
df1 = pd.DataFrame([[1, 2, 3], [4, 5, 6], [7, 8, 9]], columns=['A', 'B', 'C'])
```

2) 字典 (键为列名)

```
df2 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]})
```

3) 字典列表 (每个字典一行)

```
df3 = pd.DataFrame([{'A': 1, 'B': 4, 'C': 7},
                    {'A': 2, 'B': 5, 'C': 8},
                    {'A': 3, 'B': 6, 'C': 9}])
```

4) NumPy 二维数组

```
df4 = pd.DataFrame(np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]), columns=['A', 'B', 'C'])
print(df1)
print(df2)
```

```
   A  B  C
0  1  2  3
1  4  5  6
2  7  8  9
   A  B  C
0  1  4  7
1  2  5  8
2  3  6  9
```

字典创建时键作为**列名**, 各列等长; 字典列表创建时每个字典是**一行**。

1.3.2 从文件读取

pandas 提供了多种读取接口

```
# df_csv = pd.read_csv('data.csv')
```

```
# df_excel = pd.read_excel('data.xlsx')
```

```
# df_json = pd.read_json('data.json')
```

读取 Excel 需要 `openpyxl` (.xlsx) 或 `xlrd` (.xls); CSV 是课程中最常用的交换格式。

1.3.3 访问数据

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]})
print(df['A'])           # 访问一列 (返回 Series)
print(df[['A', 'B']])    # 访问多列 (返回 DataFrame)
print(df.iloc[0])        # 第一行
print(df.iloc[0, 1])     # 第 0 行第 1 列
print(df.loc[0, 'A'])    # 标签定位
```

```
print(df.iloc[0:2])    # 行切片
```

```
0    1
1    2
2    3
Name: A, dtype: int64
   A  B
0  1  4
1  2  5
2  3  6
A    1
B    4
C    7
Name: 0, dtype: int64
4
1
   A  B  C
0  1  4  7
1  2  5  8
```

1.3.4 修改与属性

```
df.loc[0, 'A'] = 10    # 按标签改单值
print(df.shape, list(df.columns), list(df.index))
(3, 3) ['A', 'B', 'C'] [0, 1, 2]
```

1.3.5 排序与布尔筛选

```
df = pd.DataFrame({'A': [1, 2, 3, 4, 5],
                   'B': [10, 20, 30, 40, 50],
                   'C': ['a', 'b', 'a', 'b', 'c']})
```

布尔索引: 注意每个条件用括号括起来

```
f1 = df[(df['A'] > 2) & (df['B'] < 40)]
f2 = df[(df['A'] > 3) | (df['C'] == 'b')]
f3 = df[~(df['A'] > 3)]
print(f1)
print(f2)
print(f3)
print(df.sort_values('B', ascending=False))
```

```
   A  B  C
2  3 30  a
   A  B  C
1  2 20  b
3  4 40  b
4  5 50  c
   A  B  C
0  1 10  a
1  2 20  b
2  3 30  a
   A  B  C
4  5 50  c
3  4 40  b
```

```
2 3 30 a
1 2 20 b
0 1 10 a
```

布尔组合务必加括号: (cond1) & (cond2), 否则 &| 的优先级会得到错误结果。

1.3.6 合并与连接

```
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
print(pd.concat([df1, df2], ignore_index=True))

left = pd.DataFrame({'key': ['a', 'b', 'c'], 'x': [1, 2, 3]})
right = pd.DataFrame({'key': ['b', 'c', 'd'], 'y': [20, 30, 40]})
print(pd.merge(left, right, on='key', how='left'))
```

	A	B
0	1	3
1	2	4
2	5	7
3	6	8

	key	x	y
0	a	1	NaN
1	b	2	20.0
2	c	3	30.0

concat 沿轴拼接; merge 按键连接 (how='inner'/'left'/'right'/'outer')。左连接时右表没有匹配的位置为 NaN。

1.4 日期时间索引

1.4.1 date_range 生成索引

```
dr = pd.date_range(start='2023-01-01', end='2023-01-05')
print(dr)

hr_dr = pd.date_range(start='2023-01-01 00:00', periods=24, freq='h')
print(hr_dr[:3])

tz_dr = pd.date_range(start='2023-01-01', periods=5, freq='D', tz='UTC')
print(tz_dr)
```

```
DatetimeIndex(['2023-01-01', '2023-01-02', '2023-01-03', '2023-01-04', '2023-01-05'],
              dtype='datetime64[ns]', freq='D')
DatetimeIndex(['2023-01-01 00:00:00', '2023-01-01 01:00:00', '2023-01-01 02:00:00'],
              dtype='datetime64[ns]', freq='h')
DatetimeIndex(['2023-01-01 00:00:00+00:00', '2023-01-02 00:00:00+00:00',
              '2023-01-03 00:00:00+00:00', '2023-01-04 00:00:00+00:00',
              '2023-01-05 00:00:00+00:00'],
              dtype='datetime64[ns, UTC]', freq='D')
```

1.4.2 用日期索引筛选

```
df_time = pd.DataFrame(index=dr, data={'A': range(5)})
```

```
print(df_time)
print(df_time[df_time.index >= pd.Timestamp('2023-01-03')])
```

```
      A
2023-01-01  0
2023-01-02  1
2023-01-03  2
2023-01-04  3
2023-01-05  4
      A
2023-01-03  2
2023-01-04  3
2023-01-05  4
```

常见误区

1. **s[0] 取位置**：整数标签与位置歧义；位置访问用 `s.iloc[0]`。
2. **标签切片是闭区间**：`s['a':'c']` 含 `c`，不是 Python 列表的开区间。
3. **Series 对齐产生 NaN**：不同索引相加缺失处为 NaN，不是报错。
4. **布尔条件不加括号**：`df[(df.A>2) & (df.B<40)]` 必须加括号。
5. **df['A'] 与 df[['A']] 不同**：前者返回 Series，后者返回单列 DataFrame。
6. **merge 忘写 on / how**：可能得到笛卡尔积或未预期的连接方式。

思考题

1. `df.loc[0]` 与 `df.iloc[0]` 在索引为 `['r0', 'r1', ...]` 时行为有何不同？
2. 为什么两个 Series 相加会产生 NaN？如何避免（如 `fillna(0)`）？
3. `s['a':'c']` 是视图还是拷贝？修改切片是否影响原 Series？
4. `pd.concat` 与 `pd.merge` 的适用场景分别是什么？

动手练习（详见 lab）

- 创建 5 行学生成绩 DataFrame，用 `.loc/.iloc` 各取 3 个值；
- 用布尔索引筛出“语文 > 80 且数学 > 90”的行；
- 造一个含重复行的 DataFrame，用 `drop_duplicates` 去重；
- 用 `date_range` 生成 30 天日期索引，并按日期筛选出后 10 天。

延伸阅读

- Pandas 官方 10 分钟入门：[链接](#)
- Pandas 官方“索引与选择数据”：[链接](#)
- Python Data Science Handbook 第 3 章（Pandas）：[链接](#)

02 数据分析：清洗、规约、统计、分组与时间序列

本节对应原版 4.2 的内容，并增补学习目标、常见误区、思考题与延伸阅读。

本节目标

- 会识别并处理**重复行** (drop_duplicates);
- 会处理**缺失值** (isna/fillna/dropna/ffill/bfill);
- 会用 **IQR** 识别异常值并替换;
- 会做 **Min-Max 归一化** 与 **Z-Score 标准化**;
- 会用 describe、聚合函数做统计描述;
- 会用 groupby 与 pivot_table 做分组与透视;
- 会用 resample 与 rolling 处理时间序列。

先修

- 01 基础数据结构 (Series/DataFrame 创建、索引、筛选); NumPy 基础。

官方文档 / 参考入口

- Pandas 缺失值指南: [链接](#)
- Pandas 分组 (groupby): [链接](#)
- Pandas 透视表: [链接](#)
- Pandas 时间序列: [链接](#)

2.1 重复、缺失与异常值

2.1.1 处理重复

```
import pandas as pd
import numpy as np
np.set_printoptions(precision=4, suppress=True)
pd.set_option('display.width', 100)
pd.set_option('display.max_columns', 20)

df = pd.DataFrame({'A': [1, 2, 2, 3, 4],
                   'B': [10, 20, 20, 30, 40],
                   'C': ['x', 'y', 'y', 'z', 'w']})

print(df)
print('重复行数:', df.duplicated().sum())
print(df.drop_duplicates())
print(df.drop_duplicates(subset=['A', 'B'], keep='last'))
```

	A	B	C
0	1	10	x
1	2	20	y
2	2	20	y
3	3	30	z

```

4  4  40  w
重复行数: 1
   A   B   C
0  1  10  x
1  2  20  y
3  3  30  z
4  4  40  w
   A   B   C
0  1  10  x
2  2  20  y
3  3  30  z
4  4  40  w

```

keep='first' (默认) 保留首次出现, keep='last' 保留最后一次; subset 指定判断重复的列。

2.1.2 处理缺失值

```

df = pd.DataFrame({'A': [1, np.nan, 3, 4, np.nan],
                   'B': [10, 20, np.nan, 40, 50]})

print(df)
print(df.isna())
print(df.isna().sum())
print('--- dropna ---')
print(df.dropna())
print('--- fillna(0) ---')
print(df.fillna(0))
print('--- ffill ---')
print(df.ffill())
print('--- bfill ---')
print(df.bfill())

   A   B
0  1.0 10.0
1  NaN 20.0
2  3.0  NaN
3  4.0 40.0
4  NaN 50.0

   A   B
0  False False
1   True  False
2  False  True
3  False False
4   True  False
A      2
B      1
dtype: int64
--- dropna ---
   A   B
0  1.0 10.0
3  4.0 40.0
--- fillna(0) ---
   A   B
0  1.0 10.0
1  0.0 20.0
2  3.0  0.0

```

```

3  4.0  40.0
4  0.0  50.0
--- ffill ---
      A      B
0  1.0  10.0
1  1.0  20.0
2  3.0  20.0
3  4.0  40.0
4  4.0  50.0
--- bfill ---
      A      B
0  1.0  10.0
1  3.0  20.0
2  3.0  40.0
3  4.0  40.0
4  NaN  50.0

```

注意新版 pandas 推荐 `ffill()/bfill()` 替代已弃用的 `fillna(method=...)`。也可以用 `df.fillna(df.median())`、`df.dropna(thresh=2)`：

```

med = df.median()
print(med)
print(df.fillna(med))
print('--- dropna(thresh=2) ---')
print(df.dropna(thresh=2))

```

```

A      3.0
B     30.0
dtype: float64
      A      B
0  1.0  10.0
1  3.0  20.0
2  3.0  30.0
3  4.0  40.0
4  3.0  50.0
--- dropna(thresh=2) ---
      A      B
0  1.0  10.0
3  4.0  40.0

```

2.1.3 用 IQR 识别异常值

```

df = pd.DataFrame({'Value': [1, 2, 3, 4, 100]})
Q1 = df['Value'].quantile(0.25)
Q3 = df['Value'].quantile(0.75)
IQR = Q3 - Q1
print('Q1, Q3, IQR:', Q1, Q3, IQR)
mask = (df['Value'] < Q1 - 1.5 * IQR) | (df['Value'] > Q3 + 1.5 * IQR)
print('outlier mask:', mask.tolist())
print('异常值:', df.loc[mask, 'Value'].tolist())

# 用中位数替换
df.loc[mask, 'Value'] = df['Value'].median()
print(df)

```

Q1, Q3, IQR: 2.0 4.0 2.0

outlier mask: [False, False, False, False, True]

异常值: [100]

	Value
0	1
1	2
2	3
3	4
4	3

处理异常值要谨慎：异常可能携带重要信息（如真实故障/异常记录），应在了解数据背景后再决定删除、替换或保留。

2.2 数据规约

2.2.1 Min-Max 归一化

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

```
df = pd.DataFrame({'feature': [1, 2, 3, 4, 5]})
df['feature_norm'] = (df['feature'] - df['feature'].min()) / (df['feature'].max() -
df['feature'].min())
print(df)
```

	feature	feature_norm
0	1	0.00
1	2	0.25
2	3	0.50
3	4	0.75
4	5	1.00

2.2.2 Z-Score 标准化

$$Z = \frac{X - \mu}{\sigma}$$

```
df['feature_std'] = (df['feature'] - df['feature'].mean()) / df['feature'].std()
print(df)
print('z 均值 (≈0):', df['feature_std'].mean(), 'z 标准差 (≈1):', df['feature_std'].std())
```

	feature	feature_norm	feature_std
0	1	0.00	-1.264911
1	2	0.25	-0.632456
2	3	0.50	0.000000
3	4	0.75	0.632456
4	5	1.00	1.264911

z 均值 (≈0): 4.4408920985006264e-17 z 标准差 (≈1): 1.0

若某列标准差为 0（所有值相同），Z-Score 会除以 0；实际中可加一个小常数 `eps` 或跳过该列。
Min-Max 依赖数据范围，Z-Score 不依赖具体范围，更适合近似正态的分布。

2.3 统计描述

```
df = pd.DataFrame({'A': [1, 2, 3, 4, 5],
                   'B': [10, 20, 30, 40, 50],
                   'C': ['a', 'b', 'c', 'd', 'e']})
print(df.describe())
print(df.describe(include='all'))
```

```

      A      B
count  5.000000  5.000000
mean   3.000000  30.000000
std    1.581139  15.811388
min    1.000000  10.000000
25%    2.000000  20.000000
50%    3.000000  30.000000
75%    4.000000  40.000000
max     5.000000  50.000000

      A      B      C
count  5.000000  5.000000  5
unique      NaN      NaN  5
top         NaN      NaN  a
freq         NaN      NaN  1
mean   3.000000  30.000000 NaN
std    1.581139  15.811388 NaN
min    1.000000  10.000000 NaN
25%    2.000000  20.000000 NaN
50%    3.000000  30.000000 NaN
75%    4.000000  40.000000 NaN
max     5.000000  50.000000 NaN
```

.describe() 默认只统计数值列; include='all' 会补充对象列的 unique/top/freq.

2.4 分组分析 (groupby)

```
df = pd.DataFrame({'Category': ['A', 'B', 'A', 'B', 'C', 'A'],
                   'Value': [10, 15, 14, 20, 5, 11]})
print(df.groupby('Category')['Value'].mean())
print(df.groupby('Category').agg({'Value': ['mean', 'sum', 'count']}))
```

```
Category
A      11.666667
B      17.500000
C       5.000000
Name: Value, dtype: float64
      Value
      mean sum count
Category
A      11.666667  35     3
B      17.500000  35     2
C       5.000000   5     1
```

groupby 后可使用 agg 传入列表 (多列多函数) 或字典 (列-> 函数)。

案例卡 2: groupby 分组统计——按班级算平均分并排名

目标

用 groupby 把学生成绩按“班级”分组，算出各班平均分；再用 .sort_values 排序、.rank 得到排名。同时给每个学生算“在班级内”的名次，理解 groupby(...).rank() 与“整组排名”的区别。

代码

```
import numpy as np
import pandas as pd
pd.set_option('display.width', 100)
pd.set_option('display.max_columns', 20)
np.set_printoptions(precision=4, suppress=True)

df = pd.DataFrame({
    '班级': ['一班', '一班', '一班', '二班', '二班', '二班', '三班', '三班', '三班'],
    '姓名': ['张三', '李四', '王五', '赵六', '钱七', '孙八', '周九', '吴十', '郑十一'],
    '分数': [88, 92, 75, 80, 85, 78, 95, 90, 82],
})

print('各班平均分（降序）：')
mean_by_class = df.groupby('班级')['分数'].mean().sort_values(ascending=False)
print(mean_by_class)

print('各班平均分排名（第 1 名 1.0）：')
print(mean_by_class.rank(ascending=False))

print('每人班级内排名（分数降序，min 法）：')
df['班级排名'] = df.groupby('班级')['分数'].rank(ascending=False, method='min')
print(df.sort_values(['班级', '班级排名']))
```

运行结果（已运行核验）

各班平均分（降序）：

```
班级
三班    89.0
一班    85.0
二班    81.0
```

Name: 分数, dtype: float64

各班平均分排名（第 1 名 1.0）：

```
班级
三班    1.0
一班    2.0
二班    3.0
```

Name: 分数, dtype: float64

每人班级内排名（分数降序，min 法）：

	班级	姓名	分数	班级排名
1	一班	李四	92	1.0
0	一班	张三	88	2.0
2	一班	王五	75	3.0
6	三班	周九	95	1.0
7	三班	吴十	90	2.0
8	三班	郑十一	82	3.0

```
4  二班  钱七  85   1.0
3  二班  赵六  80   2.0
5  二班  孙八  78   3.0
```

讲解

一句话解释：groupby 把表格按“班级”切成几堆，每堆算一个平均分；rank 在旁边写上“谁排第几”。

- `df.groupby('班级')['分数']` 表示“按班级分组，只看分数这一列”；
- `.mean()` 对每组求平均，返回以“班级”为索引的 Series；
- `.sort_values(ascending=False)` 让平均分从高到低排列；
- `.rank(ascending=False)` 给这个 Series 打排名：分数越高，名次数字越小（第 1 名是 1.0）；
- `df.groupby('班级')['分数'].rank(ascending=False, method='min')` 是“在每个班级内部给每个学生排名”，结果不会把行数压缩，而是新增一列 班级排名；
- `method='min'` 表示并列时取“最小的名次”（如两人并列第 1，则都记 1，下一名从 3 开始）。

主要用法 / API

需求	写法	说明
按一列分组	<code>df.groupby('班级')</code>	返回 GroupBy 对象
分组后对某列求平均	<code>df.groupby('班级')['分数'].mean()</code>	返回 Series
分组后多列多函数	<code>df.groupby('班级')['分数'].agg(['mean', 'sum', 'count'])</code>	返回 DataFrame
对结果排序	<code>.sort_values(ascending=False)</code>	或对 DataFrame <code>sort_values('分数', ascending=False)</code>
对结果排名	<code>.rank(ascending=False)</code>	分数越高，名次数字越小
组内排名	<code>df.groupby('班级')['分数'].rank(ascending=False)</code>	新增列，不压缩行数

常见错误

- 以为 `groupby('班级')['分数'].mean()` 返回普通 DataFrame，其实默认返回 Series；
- 对未排序的 Series 直接 `.rank()` 结果仍按原索引排列，应先 `.sort_values()` 再排名；
- `groupby(...).rank()` 默认 `method='average'`，并列时会得到 1.5 之类的小数；想要“并列同名次”用 `method='min'`；
- 用 `mean().reset_index()` 之后才能像普通 DataFrame 一样再排序/合并。

拓展 / 跟练

1. 改用 `df.groupby('班级')['分数'].agg(['mean', 'std', 'count'])`，并把结果画成条形图。
2. 加一列 科目，用 `pivot_table(values='分数', index='班级', columns='科目', aggfunc='mean')` 得到“班级 × 科目”平均分表。
3. 用 `df.groupby('班级')['分数'].transform('mean')` 给每个学生补一列“班级平均分”，体会 `transform` 与 `agg` 的差别。
4. 把 `df` 改成“一行一人”的长表（含学号），先 `groupby('姓名')['分数'].sum()` 算总分，再做班内排名。

2.5 透视表 (pivot_table)

```
df = pd.DataFrame({'Category': ['A', 'A', 'B', 'B', 'C'],
                  'Year': [2020, 2020, 2021, 2021, 2022],
                  'Value': [5, 7, 3, 4, 2]})
pt = df.pivot_table(values='Value', index='Category', columns='Year',
                    aggfunc='mean', fill_value=0)
print(pt)
```

Year	2020	2021	2022
Category			
A	6.0	0.0	0.0
B	0.0	3.5	0.0
C	0.0	0.0	2.0

`pivot_table` 会自动聚合重复项；`fill_value=0` 填充缺失交叉格。

2.6 时间序列：resample 与 rolling

```
idx = pd.date_range('2023-01-01', periods=12, freq='D')
ts = pd.Series(np.arange(12, dtype=float), index=idx)
print(ts.resample('W').mean())      # 按周重采样取均值
print(ts.rolling(3).mean())        # 3 天滑动平均
```

2023-01-01	0.0
2023-01-08	4.0
2023-01-15	9.5

Freq: W-SUN, dtype: float64

2023-01-01	NaN
2023-01-02	NaN
2023-01-03	1.0
2023-01-04	2.0
2023-01-05	3.0
2023-01-06	4.0
2023-01-07	5.0
2023-01-08	6.0
2023-01-09	7.0
2023-01-10	8.0
2023-01-11	9.0
2023-01-12	10.0

Freq: D, dtype: float64

`resample` 把高频率数据聚合成低频率（如日→周/月）；`rolling` 是在每个时间点取滑动窗口统计。

案例卡 3：缺失值与异常值清洗——IQR 替换 + 时间序列重采样

目标

把一条 14 天的日度数据先“清洗”：`ffill()` 前向填充缺失值，再用 IQR 找出异常值并用中位数替换；最后把它重采样成周均（`resample('W')`）并计算 3 天滑动平均（`rolling(3)`）。这是“数据清洗 → 时间聚合”的典型流程。

代码

```
import numpy as np
import pandas as pd
pd.set_option('display.width', 100)
pd.set_option('display.max_columns', 20)
np.set_printoptions(precision=4, suppress=True)

idx = pd.date_range('2023-01-01', periods=14, freq='D')
vals = [10, 11, 12, np.nan, 14, 15, 16, 17, 18, 19, 20, 21, 22, 120]
s = pd.Series(vals, index=idx)

print('原始 Series:')
print(s)

print('前向填充 (ffill) 后:')
s = s.fffll()
print(s)

Q1 = s.quantile(0.25)
Q3 = s.quantile(0.75)
IQR = Q3 - Q1
lo, hi = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
mask = (s < lo) | (s > hi)

print('Q1, Q3, IQR, 下界, 上界:')
print(round(Q1, 4), round(Q3, 4), round(IQR, 4), round(lo, 4), round(hi, 4))
print('异常值:', s[mask].tolist())

print('用中位数替换异常值后:')
s.loc[mask] = s.median()
print(s)

print('周均 (resample W):')
print(s.resample('W').mean().round(4))

print('3 天滑动平均 (rolling 3):')
print(s.rolling(3).mean().round(4))
```

运行结果 (已运行核验)

```
原始 Series:
2023-01-01    10.0
2023-01-02    11.0
2023-01-03    12.0
2023-01-04     NaN
2023-01-05    14.0
2023-01-06    15.0
2023-01-07    16.0
2023-01-08    17.0
2023-01-09    18.0
2023-01-10    19.0
2023-01-11    20.0
2023-01-12    21.0
```

```
2023-01-13      22.0
2023-01-14     120.0
Freq: D, dtype: float64
前向填充 (ffill) 后:
2023-01-01      10.0
2023-01-02      11.0
2023-01-03      12.0
2023-01-04      12.0
2023-01-05      14.0
2023-01-06      15.0
2023-01-07      16.0
2023-01-08      17.0
2023-01-09      18.0
2023-01-10      19.0
2023-01-11      20.0
2023-01-12      21.0
2023-01-13      22.0
2023-01-14     120.0
Freq: D, dtype: float64
Q1, Q3, IQR, 下界, 上界:
12.5 19.75 7.25 1.625 30.625
异常值: [120.0]
用中位数替换异常值后:
2023-01-01      10.0
2023-01-02      11.0
2023-01-03      12.0
2023-01-04      12.0
2023-01-05      14.0
2023-01-06      15.0
2023-01-07      16.0
2023-01-08      17.0
2023-01-09      18.0
2023-01-10      19.0
2023-01-11      20.0
2023-01-12      21.0
2023-01-13      22.0
2023-01-14      16.5
Freq: D, dtype: float64
周均 (resample W):
2023-01-01      10.0000
2023-01-08      13.8571
2023-01-15      19.4167
Freq: W-SUN, dtype: float64
3 天滑动平均 (rolling 3):
2023-01-01         NaN
2023-01-02         NaN
2023-01-03      11.0000
2023-01-04      11.6667
2023-01-05      12.6667
2023-01-06      13.6667
2023-01-07      15.0000
2023-01-08      16.0000
2023-01-09      17.0000
2023-01-10      18.0000
```

```
2023-01-11    19.0000
2023-01-12    20.0000
2023-01-13    21.0000
2023-01-14    19.8333
Freq: D, dtype: float64
```

讲解

一句话解释：先把“空缺”和“离谱的数值”补好，再把一天一天的数值聚成一周一周，并画一条平滑的 3 天平均。

- 最后一天 120 是人为放的异常值，正常数据应在 10–22 之间；
- `s.ffmpeg()` 用“上一个有效值”填充缺失：1 月 4 日缺，就用 1 月 3 日的 12 填上；
- `quantile(0.25) / quantile(0.75)` 是下/上四分位数； $IQR = Q3 - Q1$ ；
- IQR 规则： $[Q1 - 1.5 * IQR, Q3 + 1.5 * IQR]$ 之外视为异常。本例下界 1.625、上界 30.625，120 被标记为异常；
- `s.loc[mask] = s.median()` 把异常值替换成全体中位数 16.5；
- `s.resample('W').mean()` 把日度数据按“周”（W-SUN，周日为一周结束）聚合，输出每周均值；
- `s.rolling(3).mean()` 在每个日期取连续 3 天窗口求均值，所以前 2 个位置是 NaN。

主要用法 / API

需求	写法	说明
前向填充	<code>s.ffmpeg()</code>	用上一个有效值补缺失
后向填充	<code>s.bffmpeg()</code>	用下一个有效值补缺失
四分位数	<code>s.quantile(0.25) / s.quantile(0.75)</code>	下/上四分位
IQR 异常判据	<code>s < Q1 - 1.5*IQR</code> 或 <code>s > Q3 + 1.5*IQR</code>	布尔掩码
替换异常值	<code>s.loc[mask] = s.median()</code>	用中位数覆盖
降频聚合	<code>s.resample('W').mean()</code>	日 -> 周均值
滑动窗口	<code>s.rolling(3).mean()</code>	3 天移动平均

常见错误

- 没先 `s.isna().sum()` 看缺失位置，就盲目 `ffmpeg()`，可能漏掉“开头就缺”的问题；
- IQR 中的 1.5 是常用倍数，不是固定不变；换成 3 会更宽松、1 更严格；
- `resample('W')` 是**聚合**，会减少行数；`rolling(3)` 是**滑动窗口**，行数不变（前 2 行为 NaN）；
- 用 `s.loc[mask] = ...` 条件赋值前，确认 `mask` 是与 `s` 等长的布尔 Series，否则会因标签不匹配出错。

拓展 / 跟练

1. 把“替换为中位数”换成 `s.clip(Q1 - 1.5*IQR, Q3 + 1.5*IQR)` 的“截断”法，比较结果。
2. 把数据改成 60 天，分别用 `resample('M').mean()` 与 `rolling(7).mean()` 观察差异。
3. 用 `s.plot()` 画原始、清洗后、周均、滑窗四条线（记得配置中文字体）。
4. 试试旧写法 `s.fillna(method='ffmpeg')`，观察弃用提示，体会新版 `s.ffmpeg()` 的推荐。

2.7 apply vs 向量化（性能提示）

```
import time
```

```
rng = np.random.default_rng(0)
s = pd.Series(rng.normal(size=200_000))

t0 = time.perf_counter(); r1 = s.apply(lambda x: x * 2 + 1); t1 = time.perf_counter()
r2 = s * 2 + 1; t2 = time.perf_counter()
vec_faster = (t2 - t1) < (t1 - t0)
print('结果一致:', np.allclose(r1.values, r2.values))
print('向量化比 apply 更快:', vec_faster)
```

结果一致: True

向量化比 apply 更快: True

为了让输出跨机器稳定，这里只打印“向量化是否更快”的布尔结果；实际运行中向量化通常比 apply 快数十倍。优先用列/Series 的向量化写法，只在无法向量化时才用 apply。

常见误区

1. `fillna(method='ffill')` 弃用：新版用 `df.ffill()/df.bfill()`。
2. `dropna()` 默认删所有含缺失的行：可用 `thresh` 控制保留条件。
3. `groupby` 后直接 `.mean()` 会丢掉非数值列：用 `agg` 或先选列。
4. `pivot_table` 忘了 `fill_value`：交叉格缺失变 NaN。
5. `resample` 与 `rolling` 混用：前者降频聚合，后者滑动窗口。
6. 滥用 `apply`：能用向量化就不用 `apply`。

思考题

1. `dropna(thresh=2)` 在什么时候比 `dropna()` 更合理？
2. 为什么 Z-Score 标准化后均值约为 0、标准差约为 1？
3. `groupby` 与 `pivot_table` 都能做“按班级求平均”，区别是什么？
4. `resample('W')` 与 `rolling(7)` 都得到“近似一周”的统计，含义有何不同？

动手练习（详见 lab）

- 构造含重复 + 缺失的 DataFrame，去重、填充、删除后再做 IQR 处理；
- 对 5 个数值列做 Min-Max 与 Z-Score，并检查归一化后的最大/最小值；
- 用 `groupby` 统计各班平均分、最高分、人数；
- 用 `pivot_table` 得到“班级 × 科目”的平均分矩阵；
- 对 60 天随机数据做 `resample('W').mean()` 与 `rolling(7).mean()`。

延伸阅读

- Pandas 官方“数据处理指南”：[链接](#)
- pandas 中文官方教程（镜像）：[链接](#)
- Python Data Science Handbook 第 3 章：[链接](#)

03 综合案例：学生成绩分析与可视化

本章新增综合案例，用于把 01–02 的知识串起来：**数据构造** → **清洗（缺失/重复/异常）** → **分组/透视** → **可视化**。所有代码均可在 pandas ≥ 2.0 环境直接运行；配图由 build/make_chap4_figures.py 生成（固定随机种子 42）。

案例背景

某课程有 20 名学生、3 个班、3 门科目（语文/数学/英语）。原始数据为“长表”：一行表示一名学生在某科目的一次成绩。数据中有缺失值、重复行和个别异常分数，需要先清洗，再按班级统计平均分，并画出图表供老师分析。

目标

把本章知识串成一条完整流水线：**构造数据** → **清洗（缺失/重复/异常）** → **groupby / pivot_table 统计** → **matplotlib 可视化**，并学会为配图配置中文字体、让图表保存到 images/ 目录。

统一绘图字体（避免中文缺字）：

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)
import matplotlib
matplotlib.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
matplotlib.rcParams['axes.unicode_minus'] = False
```

1. 构造数据

```
import pandas as pd
import numpy as np

rng = np.random.default_rng(42)  # 固定种子，可复现
classes = ['一班', '二班', '三班']
subjects = ['语文', '数学', '英语']
base = {'一班': 82, '二班': 78, '三班': 85}
rows = []
for i in range(1, 21):
    cls = classes[i % 3]
    for sub in subjects:
        score = rng.normal(base[cls], 6)
        rows.append({'学号': f'S{i:02d}', '姓名': f'学生{i:02d}',
                     '班级': cls, '科目': sub, '分数': round(score, 1)})
df = pd.DataFrame(rows)

# 人为制造缺失与重复
df.loc[2, '分数'] = np.nan  # 语文缺失
df.loc[5, '分数'] = np.nan  # 英语缺失
df = pd.concat([df, df.iloc[[1]].copy()], ignore_index=True)

print('原始形状:', df.shape)
print('缺失值个数:', df['分数'].isna().sum())
print('重复行数:', df.duplicated().sum())

原始形状: (61, 5)
缺失值个数: 2
```

重复行数: 1

2. 清洗

```
df = df.dropna(subset=['分数'])      # 去掉缺分数的行
df = df.drop_duplicates()           # 去掉重复行
print('清洗后形状:', df.shape)

# 用 IQR 找异常值并替换为全体中位数
Q1 = df['分数'].quantile(0.25)
Q3 = df['分数'].quantile(0.75)
IQR = Q3 - Q1
mask = (df['分数'] < Q1 - 1.5 * IQR) | (df['分数'] > Q3 + 1.5 * IQR)
print('Q1, Q3, IQR:', round(Q1, 3), round(Q3, 3), round(IQR, 3))
print('异常值行数:', mask.sum())
df.loc[mask, '分数'] = df['分数'].median()
print(df['分数'].describe().round(4))
```

清洗后形状: (58, 5)

Q1, Q3, IQR: 77.775 85.625 7.85

异常值行数: 1

```
count    58.0000
mean     81.8466
std       5.5063
min      71.8000
25%      77.7750
50%      81.9000
75%      85.4000
max      94.0000
```

Name: 分数, dtype: float64

3. 分组统计与透视

```
# 按班级求平均分
print(df.groupby('班级')['分数'].mean().round(1))

# 透视: 班级 × 科目 的平均分
piv = df.pivot_table(values='分数', index='班级', columns='科目', aggfunc='mean').round(1)
print(piv)
```

班级

一班 81.2

三班 86.3

二班 78.0

Name: 分数, dtype: float64

科目 数学 英语 语文

班级

一班 81.1 82.5 80.0

三班 85.0 86.6 87.3

二班 77.7 78.1 78.3

说明: 透视表的行按中文 Unicode 排序为 一班、三班、二班; 三班 (基础分 85) 平均最高, 二班 (基础分 78) 最低。

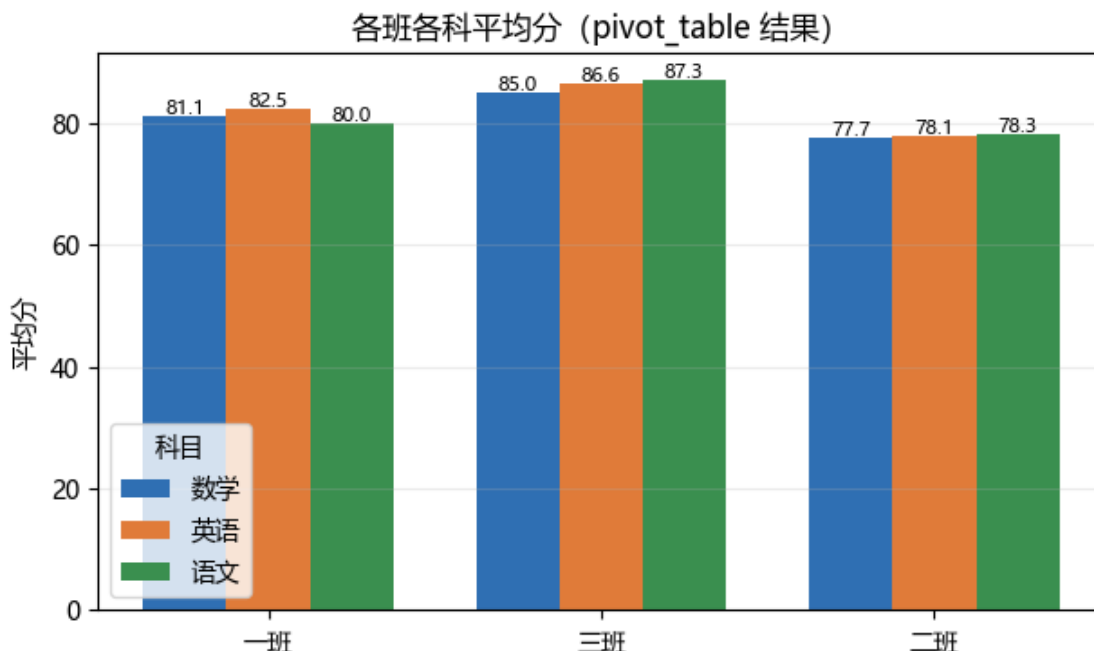


Figure 1: 各班各科平均分

讲解 (零基础)

一句话解释：先把数据变干净，再按班级“算总和/平均”，最后把结果画成图；三步里每一步都不难，关键是选对 Pandas 方法。

1. 构造数据：用 `rng = np.random.default_rng(42)` 固定随机种子，保证每次生成的“学生成绩”一样；`round(score, 1)` 让分数保留 1 位小数。
2. 清洗：`dropna(subset=['分数'])` 删掉缺分数的行；`drop_duplicates()` 删重复行；Q1/Q3/IQR 找出离群分数并用全体中位数替换。
3. 统计：`groupby('班级')['分数'].mean()` 得到各班平均分；`pivot_table(...)` 得到“班级 × 科目”矩阵，便于横向比较。
4. 可视化：`plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']` 让中文正常显示；`ax.bar`、`ax.hist`、`ax.plot` 分别画柱状、直方图与折线；`fig.savefig('images/...png')` 把图存到 `images/`。

4. 可视化

4.1 各班各科平均分柱状图

绘图代码：

```
import os
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
os.makedirs('images', exist_ok=True)
```

```
fig, ax = plt.subplots(figsize=(7.2, 4.0))
subjects = piv.columns.tolist()
```

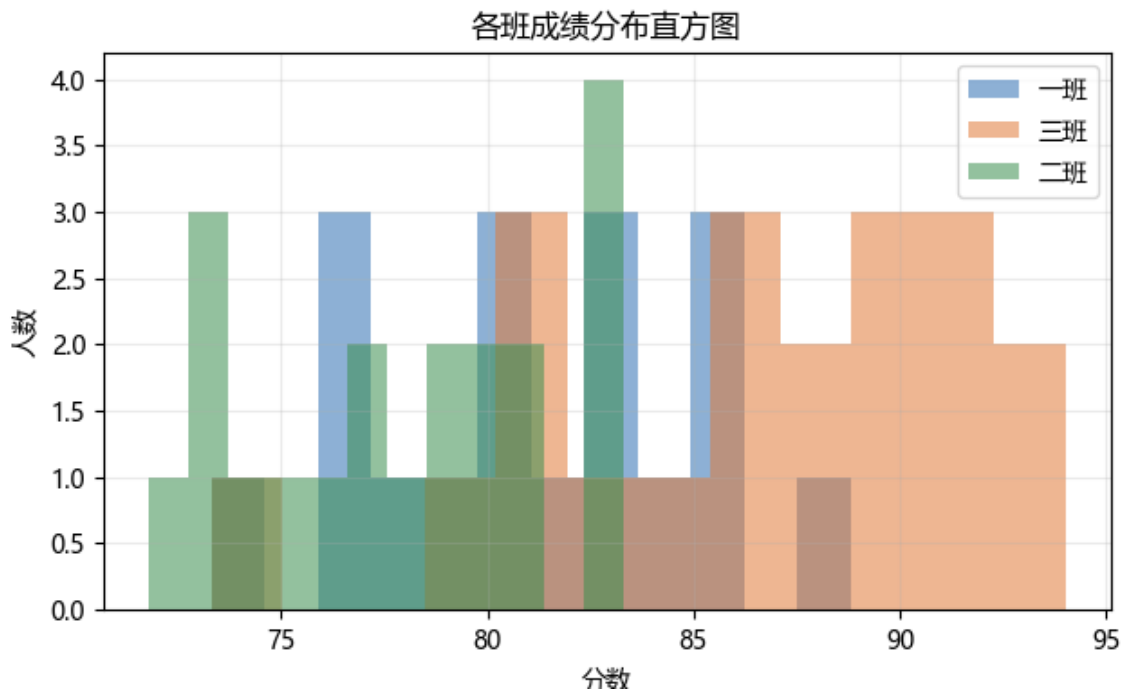


Figure 2: 各班成绩分布

```
x = np.arange(len(piv.index))
colors = ['#2f6fb3', '#e07b39', '#3a8f4f']
for j, sub in enumerate(subjects):
    vals = piv[sub].values
    ax.bar(x + (j - 1) * 0.25, vals, width=0.25, label=sub, color=colors[j])
    for xi, v in zip(x + (j - 1) * 0.25, vals):
        ax.text(xi, v + 0.3, f'{v:.1f}', ha='center', fontsize=8)
ax.set_xticks(x)
ax.set_xticklabels(piv.index)
ax.set_ylabel('平均分')
ax.set_title('各班各科平均分 (pivot_table 结果)')
ax.legend(title='科目')
ax.grid(axis='y', alpha=0.25)
fig.savefig('images/case_scores.png')
plt.close(fig)
```

4.2 各班成绩分布直方图

```
import os
os.makedirs('images', exist_ok=True)
fig, ax = plt.subplots(figsize=(7.2, 4.0))
for (cls, g), c in zip(df.groupby('班级'), colors):
    ax.hist(g['分数'], bins=12, alpha=0.55, label=cls, color=c)
ax.set_xlabel('分数')
ax.set_ylabel('人数')
ax.set_title('各班成绩分布直方图')
ax.legend()
ax.grid(alpha=0.25)
fig.savefig('images/case_distribution.png')
```

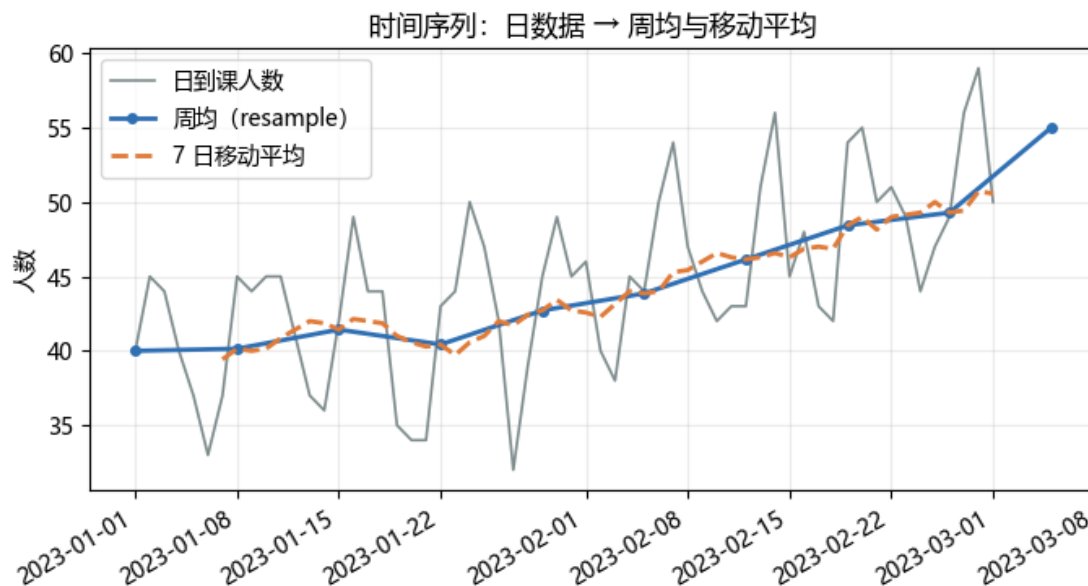


Figure 3: 时间序列

```
plt.close(fig)
```

4.3 时间序列: 日到课人数 → 周均与移动平均

```
import os
os.makedirs('images', exist_ok=True)
rng = np.random.default_rng(7)
idx = pd.date_range('2023-01-01', periods=60, freq='D')
trend = np.linspace(0, 10, 60)
weekly = 5 * np.sin(2 * np.pi * np.arange(60) / 7)
count = (40 + trend + weekly + rng.normal(0, 3, 60)).round().clip(0, 60).astype(int)
ts = pd.Series(count, index=idx)
weekly_mean = ts.resample('W').mean()
rolling7 = ts.rolling(7).mean()

fig, ax = plt.subplots(figsize=(8.0, 4.0))
ax.plot(ts.index, ts.values, color='#7f8c8d', lw=1.2, label='日到课人数')
ax.plot(weekly_mean.index, weekly_mean.values, color='#2f6fb3', marker='o', ms=4,
        lw=2, label='周均 (resample) ')
ax.plot(rolling7.index, rolling7.values, color='#e07b39', lw=2, ls='--', label='7 日移动平均')
ax.set_ylabel('人数')
ax.set_title('时间序列: 日数据 → 周均与移动平均')
ax.legend()
ax.grid(alpha=0.25)
fig.autofmt_xdate()
fig.savefig('images/case_time_series.png')
plt.close(fig)
```

5. 结论

- 清洗后共 58 条有效记录，去掉 2 个缺失、1 行重复，并用 IQR 替换了 1 个异常低分；
- 三班平均分最高 (86.3)，二班最低 (78.0)，这与我们设定的“班级基础分”一致；
- 透视表给出“班级 × 科目”的平均分，能快速定位薄弱科目；
- 时间序列图说明：resample('W') 把日数据聚合成周均，rolling(7) 得到平滑的 7 日移动平均，二者都可用于观察趋势。

主要用法 / API

需求	写法	说明
固定随机种子	<code>rng = np.random.default_rng(42)</code>	结果可复现
去缺失行	<code>df.dropna(subset=['分数'])</code>	只按指定列判断
去重复行	<code>df.drop_duplicates()</code>	默认保留首次
IQR 异常掩码	<code>(df['分数'] < Q1-1.5*IQR) (df['分数'] > Q3+1.5*IQR)</code>	布尔筛选
分组平均	<code>df.groupby('班级')['分数'].mean()</code>	返回 Series
透视表	<code>df.pivot_table(values='分数', index='班级', columns='科目', aggfunc='mean')</code>	行列交叉表格
中文绘图字体	<code>plt.rcParams['font.sans-serif']=['Microsoft YaHei', 'SimHei', 'DejaVu Sans']</code>	防止中文缺字
保存图片	<code>fig.savefig('images/case_scores.png')</code>	需先 <code>os.makedirs('images', exist_ok=True)</code>

常见错误

- 未固定随机种子导致每次运行结果不同，课堂演示/作业对不上；
- `dropna()` 不加 `subset` 会把“任何一列有缺失”的行都删掉；
- IQR 的界限为 `[Q1-1.5*IQR, Q3+1.5*IQR]`，漏乘 1.5 会把正常值当异常；
- 在临时目录运行 `fig.savefig('images/...')` 时，若 `images/` 不存在会报 `FileNotFoundError` (先 `os.makedirs('images', exist_ok=True)`)；
- matplotlib 中文乱码是没配置 `font.sans-serif`，不是数据的问题。

拓展任务

1. 改用真实 CSV（如某课程成绩表）复现；
2. 把 `groupby('班级')['分数'].agg(['mean', 'std', 'count'])` 的结果也画成条形图；
3. 对透视表加 `margins=True` 显示总计；
4. 给时间序列数据加周末标记，观察周末到课人数是否更低。

延伸阅读

- Pandas 分组与聚合文档：[链接](#)

- Pandas 透视表文档: [链接](#)
- Pandas 时间序列文档: [链接](#)
- Matplotlib 官方入门: [链接](#)

04 常见误区与技巧

本章为新增“查漏补缺”页：把学生最容易踩的坑集中列出，并给出正确的写法和性能/调试建议。

1. 高频误区速查表

#	误区 / 错误写法	正确做法	原因
1	<code>df.loc[0]</code> 想取第一行但索引是字符串	<code>df.iloc[0]</code> (位置) 或 <code>df.loc['r0']</code> (标签)	<code>loc</code> 按标签、 <code>iloc</code> 按位置
2	<code>s[0]</code> 取 Series 位置	<code>s.iloc[0]</code>	整数标签歧义，位置访问应显式 <code>iloc</code>
3	<code>df[(df.A>2) and (df.B<40)]</code>	<code>df[(df.A>2) & (df.B<40)]</code> (每个条件加括号)	<code>and/or</code> 用于 Python 布尔， <code>&/ </code> 用于逐元素布尔数组
4	<code>df['A']</code> 当“单个 DataFrame”	需要单列 DataFrame 用 <code>df[['A']]</code>	<code>df['A']</code> 返回 Series
5	<code>fillna(method='ffill')</code>	<code>df.ffill()</code>	pandas 2.x 已弃用 <code>method</code> 写法
6	<code>dropna()</code> 误删有效数据	先 <code>df.isna().sum()</code> 看缺失情况，用 <code>thresh</code> 控制	<code>dropna</code> 默认删任意含缺失的行
7	以为 <code>groupby</code> 返回 DataFrame	返回 <code>groupby</code> 对象，需 <code>.mean().agg()</code> 等	<code>groupby</code> 是“中间对象”
8	<code>pivot_table</code> 出现 NaN	加 <code>fill_value=0</code>	交叉格无数据
9	<code>resample('W')</code> 当“滑动平均”	降频聚合用 <code>resample</code> ；滑窗用 <code>rolling(7).mean()</code>	二者语义不同
10	用 <code>apply</code> 做可向量化运算	<code>s*2+1</code> 优先	<code>apply</code> 逐行 Python 调用，慢
11	修改切片后以为原数据不变	注意 Pandas 的视图/拷贝语义 (建议对新结果 <code>.copy()</code>)	部分切片是视图
12	读 Excel 报错	先 <code>pip install openpyxl</code>	缺读写引擎

2. 原理级难点：loc vs iloc

```
import pandas as pd
import numpy as np
np.set_printoptions(precision=4, suppress=True)

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s.loc['b'])      # 20 (按标签)
print(s.iloc[1])       # 20 (按位置)
```

- 标签是字符串时，`s[0]` 会歧义甚至报错；统一 `s.loc['b'] / s.iloc[1]`。
- `.loc` 切片是**闭区间** (`df.loc['a':'c']` 含 c)，`.iloc` 切片是**开区间** (`df.iloc[0:2]` 只含 0,1)。

3. 性能技巧

- **向量化优先**：能 `df['x'] * 2 + 1` 就不要 `df['x'].apply(lambda v: v*2+1)`；
- **避免在循环里逐行 append/concat**：先用列表收集再一次性 `pd.DataFrame(...)`；

- 用 `rename/astype` 前先想好副本：多数方法返回新对象；
- 大文件用 `chunksize`： `pd.read_csv('big.csv', chunksize=10_000)` 分批读；
- 时间序列避免逐元素循环：用 `resample/rolling/shift/diff`；
- 用 `pd.qcut/cut` 做分箱，避免手写条件判断。

4. 调试建议

1. 先打印 `df.head()`、`df.dtypes`、`df.shape` 确认数据形状；
2. 布尔筛选把中间掩码存成变量： `mask = (df.A > 2) & (df.B < 40)`； `df[mask]`；
3. 用 `df.isna().sum()` 检查缺失；用 `df.duplicated().sum()` 检查重复；
4. 遇到 `KeyError` 先打印 `df.columns`；
5. 数值异常用 `df.describe()` 看 min/max/分位数；
6. 随机/抽样问题**固定种子** `rng = np.random.default_rng(42)`。

5. 本章小结（自测清单）

- ☐ 能解释 Series 与 DataFrame 的区别；
- ☐ 能说出 `.loc` 与 `.iloc` 的三个不同点；
- ☐ 会写 `df[(df.A>2) & (df.B<40)]` 的多条件布尔筛选；
- ☐ 会用 `drop_duplicates/fillna/dropna/ffill` 清数据；
- ☐ 会用 IQR 找异常并替换；
- ☐ 会做 Min-Max 与 Z-Score，并说出适用场景；
- ☐ 会 `groupby/pivot_table` 做分组透视；
- ☐ 会 `resample/rolling` 处理时间序列；
- ☐ 会画 `matplotlib` 图（中文字体）并生成案例配图。

延伸阅读

- Pandas 官方 “用户指南”：[链接](#)
- Pandas 官方 “性能提升”：[链接](#)
- Python Data Science Handbook（第 3 章 Pandas）：[链接](#)